Análise de Eficiência Energética e Desempenho da Paralelização do Processamento de Imagens com Filtros Convolucionais

Enzo de Almeida Belfort Rizzi Di Chiara[1], **Luiza de Souza Ferreira**[1], **Natália Vitória Amaral do Val**[1], **Vinícius Augusto Landim Andrade**[1], **Viviane Flor Park**[1]

[1]Instituto de Ciência e Tecnologia – Universidade Federal de São Paulo (UNIFESP)
São José dos Campos – SP – Brasil

{belfort.enzo, souza.luiza, amaral.natalia, vinicius.landim, viviane.park}@unifesp.br

Abstract. *This article describes the implementation of a parallelized version of a convolutional filter algorithm and the corresponding analysis of its energy efficiency and performance. The algorithm is parallelized in two different ways – using the OpenMP library and the CUDA platform – and each version is tested with a set of images of varying dimensions. The results of the parallel implementations are compared against the sequential version of the algorithm and against each other in terms of execution time, speedup, and energy efficiency.*

Resumo. *Este artigo descreve a implementação de uma versão paralelizada de um algoritmo de filtro convolucional e sua respectiva análise de eficiência energética e desempenho. A paralelização do algoritmo é feita de duas maneiras diferentes, com a biblioteca OpenMP e a plataforma CUDA, e cada versão é testada com um conjunto de imagens com diferentes dimensões. Os resultados das implementações paralelas são comparados com a versão sequencial do algoritmo e entre si em termos de tempo de execução, speedup e eficiência energética.*

1. Introdução

O crescimento exponencial no volume de dados visuais e a necessidade de processamento em tempo real tornaram o uso de filtros convolucionais uma tarefa de alto custo computacional. Embora a paralelização seja a principal estratégia para acelerar a execução, a eficiência energética tornou-se um fator crítico no projeto de sistemas modernos. O aumento do número de *threads* e o uso intensivo de *hardware* nem sempre geram ganho proporcional de velocidade (*speedup*), podendo elevar drasticamente o consumo de energia devido a gargalos de memória e contenção de barramento.

Este trabalho tem como objetivo, portanto, avaliar o *trade-off* entre ganho de desempenho e eficiência energética na paralelização de filtros convolucionais. Para isso, são analisados diferentes algoritmos e estratégias de concorrência aplicados a imagens de variados tamanhos, mensurando o tempo de execução e o consumo energético do processador. Com isso, busca-se identificar os limites de escalabilidade em que a paralelização otimiza o desempenho mantendo a sustentabilidade energética da aplicação.

2. Fundamentação Teórica e Trabalhos Relacionados

Um filtro convolucional é um mecanismo matemático que opera uma matriz de pesos, chamada de máscara ou *kernel* (OBS: idealmente não usar "kernel"ao longo do artigo como sinônimo de filtro, já que o termo também designa o trecho de código a ser executado na GPU), sobre os píxeis de uma imagem, alterando os valores originais com base na combinação linear da vizinhança [GONZALEZ and WOODS 2010]. Devido ao seu elevado número de operações aritméticas repetitivas e à independência entre a computação de píxeis distintos, esta técnica apresenta uma elevada intensidade computacional e um alto grau de paralelismo de dados inerente, tornando-se ideal para arquiteturas concorrentes [HWU et al. 2023].

Para explorar este paralelismo, utiliza-se, por exemplo, a biblioteca OpenMP e a plataforma CUDA. O OpenMP é uma biblioteca da linguagem C que possibilita a paralelização de tarefas em sistemas de memória compartilhada. Essa biblioteca encarrega o compilador de distribuir a carga computacional em *threads* que, por sua vez, podem ser processadas em diferentes *cores* da CPU. O CUDA, por outro lado, é uma plataforma da NVIDIA que possibilita a utilização de GPUs para processamento de propósito geral. Dessa maneira, os milhares de núcleos de processamento desse componente passam a executar blocos de *threads* de forma altamente concorrente, reduzindo significativamente a latência de processamento em grandes volumes de dados.

As soluções paralelizadas são frequentemente analisadas e validadas em termos de tempo de execução, *speedup* e, mais recentemente, da sua eficiência energética. O tempo de execução mede a duração total do processamento paralelo e pode ser obtido de forma fácil com funções de auxílio das próprias plataformas utilizadas ou do terminal do sistema operacional da máquina. O *speedup*, por sua vez, é uma métrica que quantifica o ganho em tempo do algoritmo paralelizado (com tempo de execução T_P em P *cores*) em relação à versão sequencial (com tempo de execução T em 1 *core*), e é calculado pela Equação (1).

$$S_P = \frac{T}{T_P} \tag{1}$$

Se couber, descrever hierarquias de memória das GPUs e técnica de tiling

PARÁGRAFO SOBRE EFICIÊNCIA ENERGÉTICA

PARÁGRAFO SOBRE TRABALHOS RELACIONADOS

3. Metodologia e Implementação

O algoritmo sequencial usado como base para o trabalho está disponível em `parallel-conv-filter/conv_filter.c`. Estruturalmente, ele pode ser dividido em duas partes principais: o pré-processamento (função `toGray`), que transforma a imagem em tons de cinza, e a convolução (função `convolve`), de fato. Em cima desse algoritmo foram implementadas, então, as versões paralelas com OpenMP e CUDA, detalhadas nas subseções 3.1 e 3.2, respectivamente.

Os testes foram executados em cima de um banco de 100 imagens, classificadas uniformemente em 5 dimensões diferentes (CITAR DIMENSÕES). Essa execução foi

feita em um computador com um processador Intel Core i5-9400F com frequência de *clock* de 2.90GHz e 6 *cores*. O computador possui 16GB de memória RAM DDR4 e uma placa de vídeo NVIDIA GeForce GTX 1060 de 6GB.

3.1. Paralelização com OpenMP

A paralelização do algoritmo com OpenMP, disponível em `parallel-conv-filter/openmp/conv_filter.c`, foi realizada em dois momentos distintos: na parte de pré-processamento e de convolução. As duas se tornaram fortes candidatas à paralelização devido à independência entre o processamento de píxeis diferentes.

Na função de pré-processamento, foi utilizada a diretiva `#pragma omp parallel for schedule(static)`, que permite distribuir as iterações do *loop* entre as *threads* de forma equitativa. Desse modo, diferentes píxeis podem ser processados em paralelo. Já na função de convolução, foi utilizada a diretiva `#pragma omp parallel for collapse(2)`. Nesse caso, o elemento `collapse(2)` combina os dois *loops* aninhados, referentes às coordenadas x e y da imagem, em um único espaço de iterações que pode ser distribuído entre as *threads*. Assim, o processamento de píxeis da imagem de saída pode ser realizado em paralelo.

Porém, como a ferramenta de compilação utilizada não possui suporte ao elemento `collapse(2)` por este pertencer a uma versão mais recente do OpenMP, o modificador foi removido por questões de compatibilidade. Consequentemente, a função de convolução foi adaptada para utilizar a diretiva `#pragma omp parallel for schedule(static)`, distribuindo de forma equitativa as iterações do *loop* mais externo entre as *threads*.

Para chegar na versão final da implementação com OpenMP, foram realizados testes de tempo de execução das funções, comparando diferentes configurações de paralelização. A configuração apresentada foi escolhida com base nos menores tempos de execução obtidos.

3.2. Paralelização com CUDA

As paralelizações usando CUDA basearam-se nas estratégias projetadas e descritas por [HWU et al. 2023] (OBS: no modelo SBC deve-se usar `citeonline` para citações indiretas?). Dentre elas, três abordagens foram implementadas, cada uma explorando diferentes níveis da hierarquia de memória da GPU. A primeira consiste em uma abordagem ingênua, caracterizada pelo acesso extensivo de leitura e escrita diretamente na memória global. A segunda abordagem aproveita o tamanho reduzido e dos valores constantes do filtro convolucional, alocando-o na *constant memory*. Por fim, a terceira estratégia emprega a técnica de *tiling*, permitindo o uso da memória compartilhada para reduzir acessos redundantes à memória global durante o cálculo dos píxeis de saída.

3.2.1. Paralelização base

Nesta abordagem inicial, a responsabilidade de calcular cada pixel da imagem de saída é atribuída individualmente a uma *thread* distinta. Para isso, as *threads* são organizadas em um *grid* bidimensional que recobre toda a dimensão da imagem, subdividido

estruturalmente em blocos compostos por 16×16 *threads*. A identificação exata de qual elemento da matriz de saída será processado por cada *thread* baseia-se em seus índices espaciais. Essas coordenadas são calculadas combinando a dimensão e o índice do bloco com o índice interno da *thread*. Pelo código, isso pode ser feito por meio da expressão `blockIdx.x * blockDim.x + threadIdx.x` e sua respectiva correspondente no eixo `y`. A partir dessas coordenadas, o escopo de atuação de cada *thread* é definido.

Apesar dessa implementação ser funcional e simples, tal estratégia possui restrições de desempenho. O primeiro obstáculo é a divergência no fluxo de controle, que ocorre em função da necessidade de tratar as bordas da imagem, onde a janela do filtro convolucional ultrapassa os limites espaciais da matriz, exigindo verificações condicionais para evitar acessos inválidos à memória. O problema dessa abordagem reside no fato de que essas estruturas condicionais fazem com que *threads* de um mesmo *warp* sigam caminhos de execução distintos, já que algumas pularão os cálculos enquanto outras os realizarão. No modelo SIMT das GPUs, isso força a serialização das instruções, o que reduz o paralelismo.

O segundo gargalo é a ineficiência no acesso à memória. Como janelas de convolução vizinhas se sobrepõem quase inteiramente, um mesmo pixel de entrada é lido repetidamente por várias *threads*. O mesmo ocorre com os coeficientes do filtro, que são lidos por todas as *threads*. Além dessa redundância, todas os acessos são feitos diretamente na memória global, que possui alta latência e largura de banda limitada, sem qualquer reaproveitamento dos dados em memórias mais rápidas, como a memória compartilhada ou os registradores [HWU et al. 2023]. Como consequência, o desempenho passa a ser limitado pela vazão da memória, e não pela capacidade de cálculo da GPU.

3.3. Paralelização usando a *constant memory* e *caching*

Para reduzir a quantidade de acessos à memória global, uma alternativa é declarar o filtro como um elemento a ser armazenado na *constant memory* da GPU. Segundo [HWU et al. 2023], o uso desse espaço de memória é viável e arquiteturalmente ideal devido a três características inerentes ao filtro convolucional.

Primeiramente, os seus pesos numéricos permanecem inalterados durante toda a execução do *kernel*, o que respeita a restrição dessa memória ser do tipo *read-only*. Além disso, a matriz do filtro possui dimensões reduzidas, de modo que os dados ocupam apenas alguns *kilobytes*, encaixando-se no limite de capacidade de 64 KB típico dessa memória.

A terceira razão diz respeito ao funcionamento da cache associada à *constant memory*. Embora a *constant memory* esteja fisicamente na DRAM do dispositivo, assim como a memória global, seus dados são acessados por meio de caches especializadas dentro de cada SM. O aproveitamento dessas *caches* depende do padrão de acesso, e a convolução é favorável nesse aspecto. No caso, as *threads* de um mesmo *warp* operam de forma sincronizada e, como os índices usados para acessar o filtro dependem apenas dos laços de repetição, e não do índice de cada *thread*, todas elas acessam a mesma variável do filtro ao mesmo tempo. Nesse cenário, as *caches* conseguem fornecer uma grande largura de banda para atender à demanda de dados das *threads*. Além disso, como o filtro é pequeno, é possível supor que seus elementos são sempre acessados a partir da cache, e não da *constant memory* [HWU et al. 2023].

Na implementação, a principal modificação no código é a declaração do filtro em escopo global utilizando a palavra-chave `__constant__`. Devido a essa mudança, a transferência dos dados da CPU para a placa de vídeo passa a ser realizada pela função `cudaMemcpyToSymbol`, que serve justamente para sinalizar à GPU que os valores copiados não serão alterados durante a execução do *kernel*. Além disso, como o vetor do filtro adquire escopo global no dispositivo, ele pode ser acessado diretamente pelas *threads*, não sendo mais necessário passá-lo como um parâmetro na chamada da função. OBS: faz parte do escopo do artigo esses detalhes de implementação?

3.3.1. Paralelização usando *tiling*

Uma característica inerente à operação de convolução é que, para calcular o valor de dois *píxeis* de saída adjacentes, o filtro se desloca apenas uma posição, de modo que é necessária a leitura de *píxeis* de entrada repetidos, com apenas uma fração de novos dados. Nas abordagens anteriores esse problema não foi considerado, de modo que a imagem de entrada era lida da memória global, e cada reutilização de um mesmo pixel resultava em uma nova requisição a essa memória [HWU et al. 2023].

Sendo assim, a técnica de *tiling* surge como uma solução de desempenho. Essa abordagem consiste em particionar o grande volume de dados globais em subconjuntos menores, de modo a permitir sua alocação em uma memória menor e de acesso mais rápido, como a memória compartilhada (OBS: Caso colocar a definição da técnica na fundamentação teórica, remover esse trecho). Aplicando o *tiling*, o segmento da imagem necessário às *threads* de um bloco é carregado uma única vez da memória global para a memória compartilhada, e as leituras repetidas passam a ser atendidas por essa memória interna, de acesso significativamente mais rápido.

Nesta terceira implementação, a técnica de *tiling* foi aplicada na memória compartilhada, mantendo o filtro na *constant memory*. Para viabilizar essa arquitetura, uma condição estrutural é que o *tile* de entrada seja estritamente maior que o *tile* de saída. Isso se deve à necessidade de incluir margens em todas as direções com largura igual ao raio do filtro, garantindo os dados necessários para o cálculo dos *píxeis* situados nas extremidades do bloco de saída.

A respeito da implementação, como escolha de projeto, os blocos de *threads* foram configurados com as mesmas dimensões do *tile* de saída, delegando a cada *thread* o cálculo de um único píxel resultante. Contudo, pelo fato de o *tile* de entrada possuir mais elementos do que a quantidade de *threads* disponíveis no bloco, é necessário que cada *thread* transfira mais de um elemento da imagem da memória global para a compartilhada. Somente após o carregamento, seguido de uma instrução de sincronização, é que a convolução se inicia, com as *threads* consumindo os dados localmente. Como cada píxel de entrada é lido repetidas vezes por diversas *threads*, o custo da carga inicial é compensado com a redução do tráfego na memória global, maximizando o desempenho [HWU et al. 2023].

4. Resultados e Discussão

5. Conclusão

Referências

[AFIF et al. 2018] AFIF, M., SAID, Y., and ATRI, M. (2018). Efficient 2d convolution filters implementations on graphics processing unit using nvidia cuda. *International Journal of Image, Graphics and Signal Processing*, v. 8.

[CREMEL and ARRUDA 2026] CREMEL, G. and ARRUDA, L. F. (2026). High-performance parallel acceleration of image processing filters on multicore cpus using openmp. *Anais do Computer on the Beach*, v. 17.

[GONZALEZ and WOODS 2010] GONZALEZ, R. and WOODS, R. (2010). *Processamento digital de imagens*. Pearson Education do Brasil, São Paulo, 3 edition.

[HUMAIDAN et al. 2024] HUMAIDAN, B. A., ALGHOFHAILY, S., QHATHANI, M. A., S.OUDAH, and NAGY, N. (2024). Parallel image processing: Taking grayscale conversion using openmp as an example. *Journal of Computer and Communications*, v. 12.

[HWU et al. 2023] HWU, W. W., KIRK, D. B., and HAJJ, E. I. (2023). *Programming Massively Parallel Processors*. Morgan Kaufmann, 4th edition.